Work-Sharing Construct

- Work-sharing is to split up pathways through the code between threads within a team
  - Loop construct
    - #pragma omp for
  - Sections/section constructs
    - #pragma omp sections
  - Single construct
    - #pragma omp single
- Two directives to be studied
  - Do/for: concurrent loop iterations
    - Shares iterations of a loop across the group (data parallelism")
    - Partitions parallel iterations across threads
    - End of for loop: implicit barrier

```
#pragma omp for [clause list]
  /* for loop */
```

Work-Sharing Construct

- The parallel and work-sharing (except single) constructs can be combined
  - Loop construct #pragma omp parallel for
  - Sections construct #pragma omp parallel sections

- Distribute iterations in a parallel region
  - shared clause: All threads can read from and write to a shared variable.
  - private clause: Each thread has a local copy of a private variable

```
#pragma omp parallel for shared(n,a) private(i)
        for (i=0; i <n; i++)
            a[i] = i + n;
```

  - The mapping between iterations and threads can be controlled by the schedule clause. .

SPMD vs. worksharing

- A parallel construct by itself creates **a** Single Program Multiple Data.
  - each thread redundantly executes the same code.

```
#pragma omp parallel
{
        int id, i, Nthrds, istart, iend;
        id = omp_get_thread_num();
        Nthrds = omp_get_num_threads();
        istart = id * N / Nthrds;
        iend = (id+1) * N / Nthrds;
        if (id == Nthrds-1)iend = N;
        for(i=istart;I<iend;i++)  { a[i] = a[i] + b[i];}
}
```

- **Work-sharing in OpenMP**

```
for(i=0;I<N;i++)  { a[i] = a[i] + b[i];}
```

- OpenMP shortcut: Put the “parallel” and the worksharing directive on the same line

```
#pragma omp parallel for
```

OpenMP parallel region work-sharing for construct

```
#pragma omp parallel
#pragma omp for
for(i=0;I<N;i++)
 { a[i] = a[i] + b[i];}
```

Example Using for


```
#include <stdlib.h>
#include <stdio.h>
#include "omp.h"
int main()
{
 int nthreads, tid;

 omp_set_num_threads(3);
 #pragma omp parallel private(tid)
   {
    int i;
     tid = omp_get_thread_num();
 printf("Hello world from (%d)\n", tid);
 #pragma omp for
     for(i = 0; i <=4; i++)
     {
```

Write openMP code to add two vectors using parallel for

1. • Two work-sharing loops in one parallel region

```
#pragma omp parallel shared(n,a,b) private(i)
{
#pragma omp for
for (i=0; i<n; i++) a[i] = i+1;
// there is an implied barrier
#pragma omp for
for (i=0; i<n; i++) b[i] = 2 * a[i];
}
```

schedule clause

- Defines schedules for OpenMP loops.
- a specification of how iterations of associated loops are divided into contiguous non-empty subsets
  - We call each of the contiguous non-empty subsets a chunk and how these chunks are distributed to threads of the team
- The size of a chunk, denoted as chunk_size must be a positive integer

Schedule Types

- static
- dynamic
- guided
- auto
- runtime

```
#pragma omp parallel for schedule([modifier [modifier]:]kind[,chunk_size])
```

provides a hint for how iterations of the corresponding OpenMP loop should be assigned to threads

OMP_SCHEDULE

omp_set_schedule(sched_type, chunk_size);

schedule clause

- Static
  - Loop iterations are divided into pieces of size chunk
  - If chunk is not specified, the iteration are evenly) divided contiguously among the threads

Static scheduling • 16 iterations, 4 threads

Thread	0	1	2	3
*no chunk**	1-4	5-8	9-12	13-16
chunk = 2	1-2 9-10	3-4 11-12	5-6 13-14	7-8 15-16

Schedule : Static example


```
int main()
{
#pragma omp parallel for schedule(static, 3)    for (int i = 0; i < 20; i++)
        {
                printf("Thread %d is running number %d\n", omp_get_thread_num(), i);
        }
        return 0;
}
```

```
int main()
{
        omp_set_num_threads(4);
#pragma omp parallel for schedule(static, 3)    for (int i = 0; i < 20; i++)
        {
                printf("Thread %d is running number %d\n", omp_get_thread_num(), i);
        }
        return 0;
}
```



schedule clause

- OpenMP will still split task into iter_size/chunk_size chunks, but distribute trunks to threads dynamically without any specific order

#pragma omp parallel for schedule(dynamic, 1) is equivalent to #pragma omp parallel for schedule(dynamic)

#pragma omp parallel for schedule(dynamic,chunk-size)

The dynamic scheduling type is appropriate when the iterations require different computational costs

schedule clause

- **Guided :** Chunk size is dynamic while using guided method, the size of a chunk is proportional to the number of unassigned iterations divided by the number of the threads. Size of each successive chunks is decreasing
  - chunk size = max((num_of_iterations remaining / 2*num_of_threads), n)
- **runtime** The scheduling decision is deferred until runtime by the environment variable OMP_SCHEDULE. It is illegal to specify a chunk size for this clause
- **auto** The scheduling decision is made by the compiler and/or runtime system

• Compiler forms a single loop and then parallelizes this

```
#pragma omp parallel for collapse (2)
for(i=0;i< N; i++)
{
      for(j=0;j< M; j++)
      {
          foo(A,i,j);
      }
 }
```

Work sharing : Section Directive

- One thread executes one section. Each section is executed exactly once.

```
#pragma omp parallel
  #pragma omp sections
   {
    #pragma omp section
       x_calculation();
    #pragma omp section
       y_calculation();
    #pragma omp section
       z_calculation();
   }
```

- #pragma omp single (Designated section is executed by single thread only.)
- #pragma omp master : Similar to single, but code block will be executed by the master thread only.
- #pragma omp critical:

```
#pragma omp parallel sections {
 #pragma omp section
funcA();
#pragma omp section
funcB(); } /*-- End of parallel region --*/
```

Homework

1. Create a parallel version of the pi program using a parallel construct. ???

```
#include <stdio.h>
#include <stdlib.h>
#define NUM_STEPS 10000
int main( void )
{
int i;
double sum = 0.0, pi, x_i;
double step = 1.0/NUM_STEPS;
for ( i = 0; i < NUM_STEPS; i++ ) {
x_i = (i + 0.5) * step;
sum = sum + 4.0 / (1.0 + x_i * x_i);
}
pi = sum * step;
printf("Pi: %.15e\n", pi);
return 0;
}
```

Hints:

#pragma omp parallel

num_threads(...)

omp_set_num_threads(...)

omp_get_num_threads(...)

omp_get_thread_num(...)

Challenges:

split iterations of the loop among threads

create an accumulator for each thread to hold partial sums, which can later be combined to generate the global sum

```
int main()
{
   int   b[3];
   char  *cptr;
   int   i;

    cptr = malloc(1);
  #pragma omp parallel for
    for(i=0; i<3; i++)
        b[i]=i;
}
```

1. List out the private and shared variable in following code

1. We want each thread's private copy of array element x[0] to inherit the value that the shared variable was assigned in the master thread

```
x[0] = 1.0;
for(i=0; i < n; i++){
    for(j=1; j<4; j++)
      x[j]=g(i, x[j-1]);
    answer[i]=x[1]-x[3];
}
```

#pragma omp parallel for private (j) firstprivate (x)

- How do we decide which variables should be shared and which private?
  - Loop indices - private –
  - Loop temporaries - private –
  - Read-only variables - shared –
  - Main arrays - shared

Example

- Write OpenMP program to calculate the average of an array of integers.
  - The program should efficiently distribute the workload among multiple threads, and you must handle critical sections appropriately to avoid data races and ensure accurate results.

Solution:


```
// Initialize the array with values
   for (int i = 0; i < ARRAY_SIZE; ++i) {
       array[i] = i + 1;
   }
```

```
#pragma omp parallel num_threads(num_threads) shared(sum)
   {
       double local_sum = 0.0;
```

```
#pragma omp for
       for (int i = 0; i < ARRAY_SIZE; ++i) {
           local_sum += array[i];
       }
```

```
       #pragma omp critical
       {
           sum += local_sum;
       }
   }
```

```
 double average = sum /
ARRAY_SIZE;

    printf("Average: %f\n", average);
```

- Design a parallel program in C using OpenMP to perform a linear search on an array.
- The goal is to
  - efficiently search for a specific target
  - Handle critical sections appropriately to ensure the correct identification

```
#pragma omp parallel for
for (int i = 0; i < size; ++i) {
    if (arr[i] == target) {
        #pragma omp critical
        {
            result = i;  // Set result only once using a critical section
        }
    }
}
```

Reduction

- Reduction (operator: variable list):
  - Combining of local copies of a variable in different threads into a single copy at the master when threads exit.
  - Variables in variable list are implicitly private to threads.
- Operators: +, *, -, &, |, ^, &&, and || – Usage

```
#pragma omp parallel reduction(+: sums) num_threads(4)
{ /* compute local sums in each thread }
```

Example


```
sum = 0;
for (int  i = 0; i < 10; i++)
{
   sum += a[i]
}
```

#pragma omp parallel for shared(sum, a) **reduction(+: sum)**

```
sum = 0;
#pragma omp parallel for shared(sum, a) reduction(+: sum)
for ( int i = 0; i < 10; i++) {
sum += a[i] }
```

```
for( int i = 0; i < numSubdivisions; i++ )
{ double x = A + dx * (float) i;
 double f = Function( x );
sum += f;
 }
```

There are Three Ways to Make the Summing Work Correctly:

```
#pragma omp atomic
sum += f;
```

```
#pragma omp critical
sum += f;
```

```
#pragma omp parallel for shared(dx),reduction(+:sum)
```

Advantages and Disadvantages openMP

Advantages

- Shared address space provides user friendly programming
- Ease of programming
- Data sharing between threads is fast and uniform (low latency)
- Incremental parallelization of sequential code
- Leaves thread management to compiler
- Directly supported by compiler

Disadvantages

- Internal details are hidden
- Programmer is responsible for specifyingsynchronization, e.g. locks
- Cannot run across distributed memory
- Performance limited by memory architecture
- Lack of scalability between memory and CPUs
- Requires compiler which supports OpenMP

Monte Carlo to estimate PI

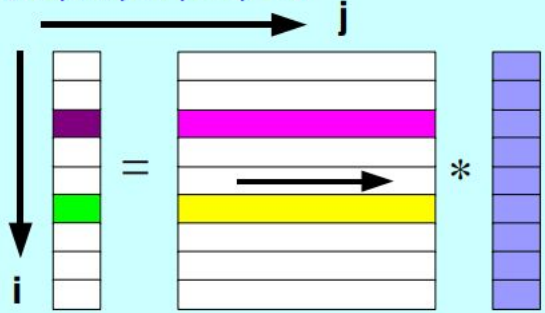

```
#include <stdlib.h>
#include <stdio.h>
#include "omp.h"
int main(int argc, char *argv[])
{
 long int i, count; // count points inside unit circle
 long int samples; // number of samples
 double pi;
 unsigned short xi[3] = {1, 5, 177}; // random number seed
 double x, y;
 samples = atoi(argv[1]);
 count = 0;
 for(i = 0; i < samples; i++)
 {
 x = erand48(xi);
 y = erand48(xi);
 if(x*x + y*y <= 1.0) count++;
```

```
#include <stdio.h>
#include <stdlib.h>
#include “omp.h”
main(int argc, char *argv[])
{
 int i, count; /* points inside the unit quarter circle */
 unsigned short xi[3]; /* random number seed */
 int samples; /*samples Number of points to Generate*/
 double x,y; /* Coordinates of points */
 double pi; /* Estimate of pi */
 samples = atoi(argv[1]);
 #pragma omp parallel
 {
 xi[0] = 1; /* set up the random seed */
 xi[1] = 1;
 xi[2] = omp_get_thread_num();
 count = 0;
 printf("I am thread %d\n", xi[2]);
 #pragma omp for firstprivate(xi) private(x,y) reduction(+:count)
 for (i = 0; i < samples; i++)  {
```

Mid-QP


```
for (i=0; i<m; i++)
{
  a[i] = 0.0;
  for (j=0; j<n; j++)
     a[i] += b[i][j]*c[j];
}
```

```
#pragma omp parallel for default(none) \
        private(i,j) shared(m,n,a,b,c)
for (i=0; i<m; i++)
{
  a[i] = 0.0;
  for (j=0; j<n; j++)
     a[i] += b[i][j]*c[j];
 }
```



Tutorial 0

1. You are given an array `shared_array` of size `ARRAY_SIZE`. Multiple threads need to concurrently increment each element of the array.
2. Design a parallel program using OpenMP to achieve this, ensuring that the increments are performed atomically to avoid race conditions.

Tutorial Exercise 1

- The SAXPY program is to add a scalar multiple of a real vector to another real vector:
- s = a*x + y.
- Provided a serial SAXPY code, parallelize it using OpenMP directives.
- Compare the performance between serial and OpenMP codes.

```
for { i = 0; i < n; i++ }
{
  y[i] = a * x[i] + y[i];
}
```

- Check total wall clock execution time versus thread numbers:
  - export OMP_NUM_THREADS=1
  - time ./saxpy
  - export OMP_NUM_THREADS=2
  - time ./saxpy
  - export OMP_NUM_THREADS=4
  - time ./saxpy
  - export OMP_NUM_THREADS=8
  - time ./saxpy

Tutorial Exercise 2

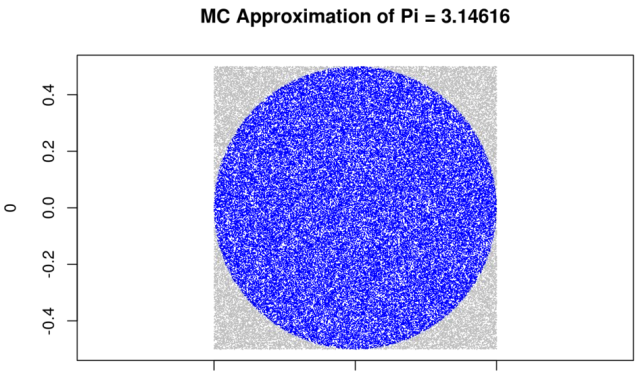
1. **Estimating the value of Pi using Monte Carlo**
    1. computational algorithms that rely on repeated random sampling to obtain numerical results

Estimation of Pi

We know that area of the square is 4r^2 unit sq while that of circle is pi*r^2.

The ratio of these two areas is as follows : pi/4

1. Generate a random point (x, y) inside a square of side 2 centered at the origin.
2. Determine whether the point falls inside the unit circle inscribed in the square by checking whether $x^2 + y^2 <= 1$.
3. Repeat steps 1 and 2 for a large number of points (e.g., 10^7).
4. Calculate the ratio of the number of points that fell inside the circle to the total number of points generated.
5. Multiply the ratio by 4 to estimate the value of pi.



Monte Carlo to estimate PI


```
#include <stdlib.h>
#include <stdio.h>
#include "omp.h"
int main(int argc, char *argv[])
{
 long int i, count; // count points inside unit circle
 long int samples; // number of samples
 double pi;
 unsigned short xi[3] = {1, 5, 177}; // random number seed
 double x, y;
 samples = atoi(argv[1]);
 count = 0;
 for(i = 0; i < samples; i++)
 {
 x = erand48(xi);
 y = erand48(xi);
 if(x*x + y*y <= 1.0) count++;
```

- Provided a serial code, parallelize it using OpenMP directives.
- Compare the performance between serial and OpenMP codes

References

1. http://openmp.org 2.
2. https://computing.llnl.gov/tutorials/openMP 3.
3. http://www.openmp.org/mp-documents/OpenMP4.0RC1_final.pdf
4. Michael J. Quinn, Parallel Programming in C with MPI and OpenMP, Mc Graw Hill, 2003.